

AWS CLOUD COMPUTING

Configuring S3 Upload-Only Access Using a Custom IAM Policy

A hands-on walkthrough of restricting an IAM user to Amazon S3 object uploads and bucket listing, while explicitly blocking object deletion.

AWS Service Used	Amazon S3 & IAM
S3 Bucket	trushti (region: us-east-1, N. Virginia)
IAM User	tushu
Custom Policy	ImageUploadNoDeletePolicy1

1. Objective

The goal of this exercise is to create an IAM user who can upload files to a specific S3 bucket and list its contents, but who cannot delete any object from that bucket. This is a common least-privilege pattern used for upload-only integrations, such as a web form or an application that only needs to write files to storage without the ability to remove them.

Permission	Purpose	Result
s3:PutObject	Allows the user to upload new objects into the bucket	Allowed
s3:ListBucket	Allows the user to view/list the objects already in the bucket	Allowed
s3:DeleteObject	Would allow removing objects from the bucket	Denied

2. Background Theory

2.1 IAM Policies

AWS Identity and Access Management (IAM) controls who can do what in an AWS account. A policy is a JSON document attached to a user, group, or role that explicitly lists which actions are Allowed or Denied on which resources. By default, an IAM user has no permissions at all — every capability must be explicitly granted. This 'deny by default' model is what makes it possible to build an upload-only user: we simply never grant the delete action, so it stays blocked.

2.2 Amazon S3 Resource Hierarchy

S3 permissions are scoped at two levels, and a policy needs to reference the correct one for each action:

- Bucket-level actions (e.g. s3:ListBucket) act on the bucket itself, so the resource ARN has no trailing path: `arn:aws:s3:::trushti`
- Object-level actions (e.g. s3:PutObject, s3:DeleteObject) act on the objects inside the bucket, so the resource ARN needs a wildcard: `arn:aws:s3:::trushti/*`

Mixing these up is one of the most common mistakes when writing S3 policies — granting PutObject on the bucket ARN (without /) or ListBucket on the object ARN (with /*) simply will not work.*

2.3 Why Omitting DeleteObject Is Enough

Because IAM evaluates permissions on an explicit allow-list basis, an action that is never mentioned anywhere in the user's attached policies is denied automatically — there is no need to write an explicit Deny statement for s3:DeleteObject. The only risk is if some other policy attached to the same user grants delete access; that would override the restriction, since an explicit Allow elsewhere would still apply unless specifically denied. This is why Part 6 of this document double-checks that the only policy attached to the user is the custom one created here.

3. Solution Architecture

At a high level, the setup looks like this:

Access flow

IAM user 'tushu' → PutObject / ListBucket (allowed) → S3 bucket 'trushti' IAM user 'tushu' ✕
DeleteObject (blocked) → S3 bucket 'trushti'

4. Part 1 — Creating the S3 Bucket

Step 1 Open the S3 console

Sign in to the AWS Management Console and search for “S3” in the top search bar, then open the S3 service.

Step 2 Create a new bucket

From the S3 dashboard, click Create bucket and choose a globally unique bucket name. In this walkthrough the bucket is named trushti, created in the US East (N. Virginia) us-east-1 region.

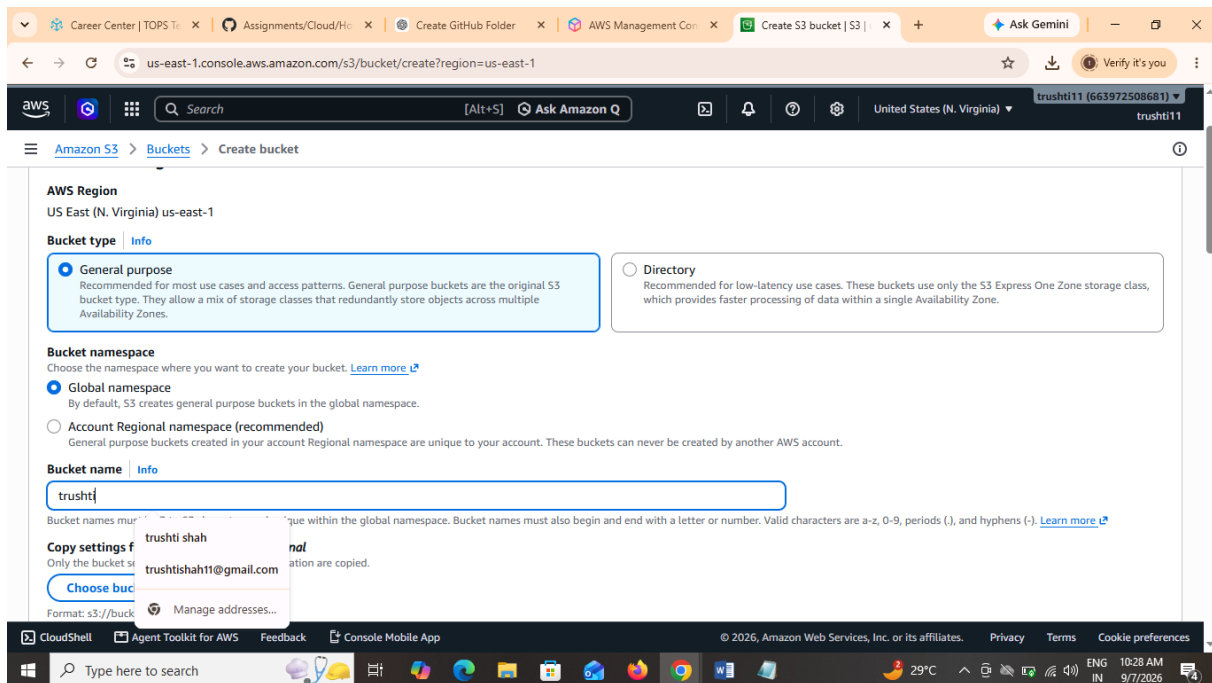


Figure 1 — Naming the bucket during creation (General purpose bucket, global namespace).

Step 3 Keep default security settings

Leave Object Ownership as “ACLs disabled / Bucket owner enforced” and keep every Block Public Access option checked — the bucket must stay private. Versioning is left Disabled since it isn't required for this exercise.

Step 4 Confirm encryption and create the bucket

Under Default encryption, Server-side encryption with Amazon S3 managed keys (SSE-S3) is used with the S3 Bucket Key enabled to reduce KMS costs. Scroll down and click Create bucket.

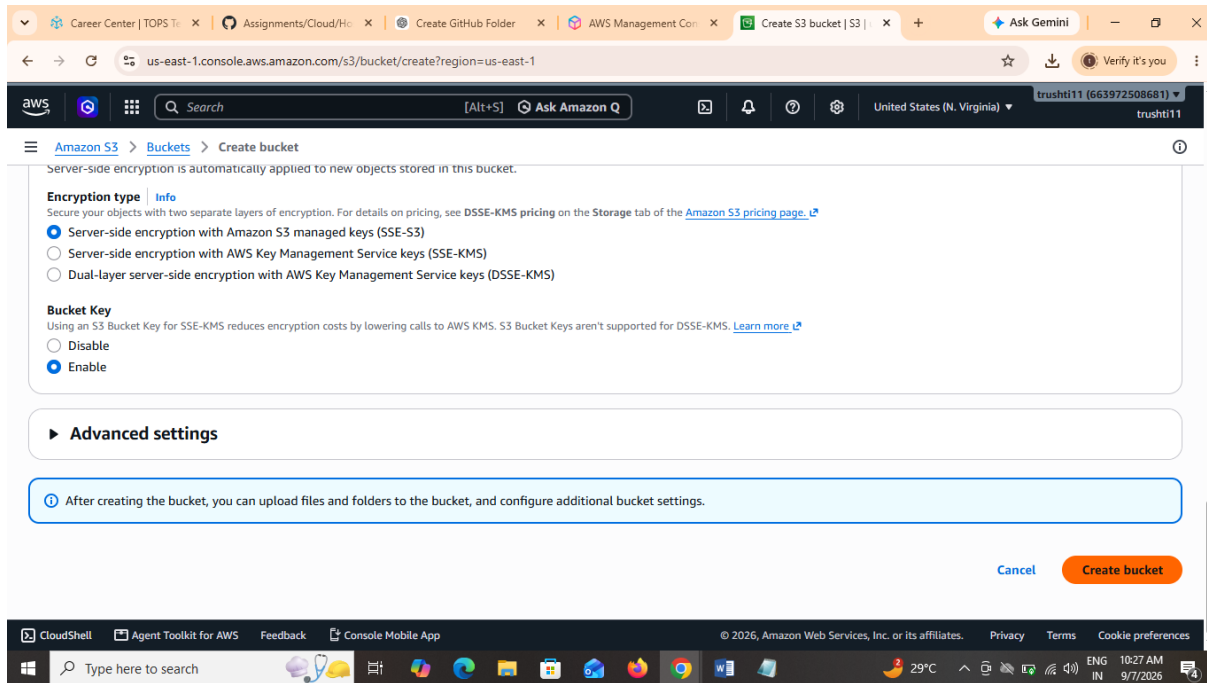


Figure 2 — Encryption settings and the final Create bucket action.

5. Part 2 — Noting the Bucket ARN

Every S3 resource has an Amazon Resource Name (ARN). For the bucket created above, the two ARNs needed for the policy are:

```
Bucket-level ARN (for ListBucket):
arn:aws:s3:::trushti

Object-level ARN (for PutObject):
arn:aws:s3:::trushti/*
```

6. Part 3 — Creating the Custom IAM Policy

Step 1 Open IAM → Policies → Create policy

Search for “IAM” in the console, open the IAM service, select Policies from the left navigation, then click Create policy.

Step 2 Switch to the JSON editor and paste the policy

Select the JSON tab, remove the placeholder statement, and paste the following policy, replacing the bucket name with the actual bucket (trushti):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowImageUpload",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::trushti/*"
    },
    {
      "Sid": "AllowBucketListing",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::trushti"
    }
  ]
}
```

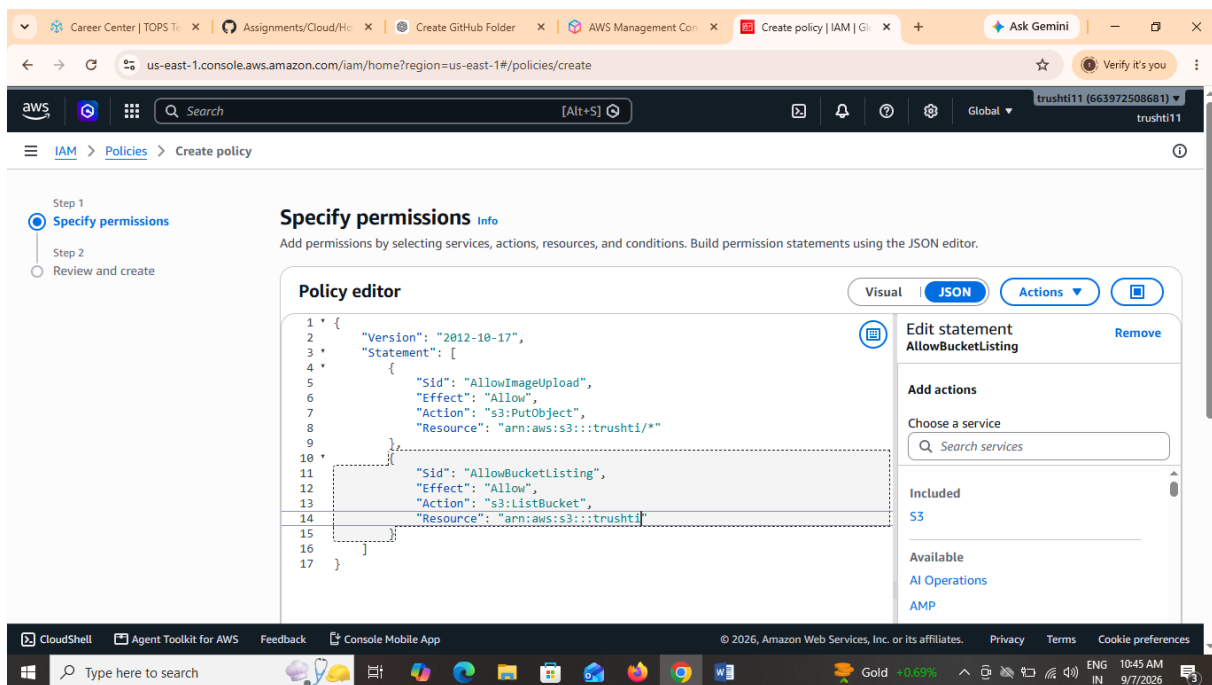


Figure 3 — Policy editor showing the PutObject and ListBucket statements scoped to the trushti bucket.

Step 3 Name and create the policy

On the review screen, name the policy ImageUploadNoDeletePolicy1 and (optionally) add a description such as “Allows users to upload objects to the S3 bucket but does not allow object deletion.” The permissions summary confirms Limited: List, Write access on the S3 service, with no Delete access listed.

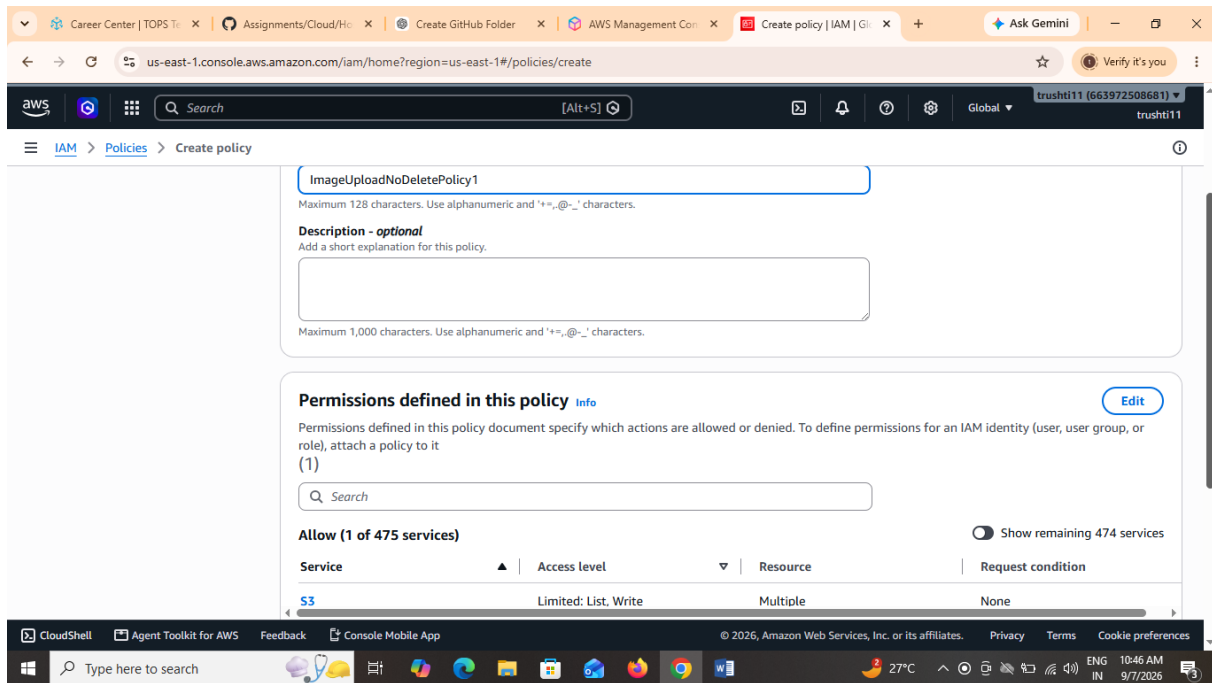


Figure 4 — Naming the policy and reviewing the permissions it grants before creation.

7. Part 4 — What Each Statement in the Policy Does

7.1 s3:PutObject

This action lets the user upload any object into the bucket — an image, a PDF, a text file, or anything else. The policy itself does not restrict the file type; it only controls the upload action. If only image files should be allowed, the policy could be extended with a condition on the object key suffix, but that refinement is outside the scope of this assignment, where basic PutObject permission is sufficient.

7.2 s3:ListBucket

This action lets the user see the list of objects already stored in the bucket (an S3 “directory listing”). Without it, the user could still upload files but would not be able to browse what is already there.

7.3 The absence of s3:DeleteObject

Notice that s3:DeleteObject never appears anywhere in the policy. Because IAM denies anything that isn't explicitly allowed, the user simply has no way to remove objects from the bucket — this is verified later in Part 9.

8. Part 5 — Creating the IAM User

Step 1 IAM → Users → Create user

From IAM, open Users and click Create user. Give the user a name — in this walkthrough the user is named tushu — and, if console sign-in is needed for testing, enable “Provide user access to the AWS Management Console” with a custom password.

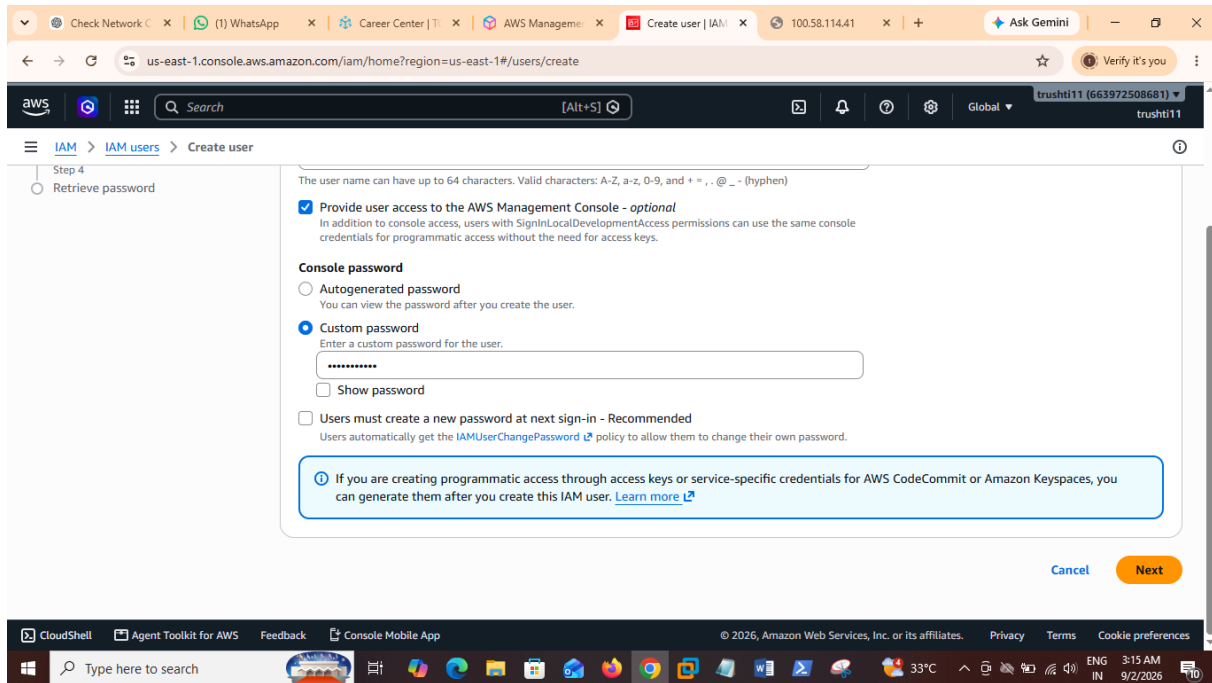


Figure 5 — Setting a custom console password while creating the IAM user.

9. Part 6 — Attaching the Policy to the User

Step 1 Choose “Attach policies directly”

On the permissions step, select Attach policies directly rather than adding the user to a group — this keeps the permission set isolated and easy to audit for this exercise.

Step 2 Search for and select the custom policy

Search for ImageUploadNoDeletePolicy1 in the policy list and check it. This is the only policy attached to this user, which matters — see the note below.

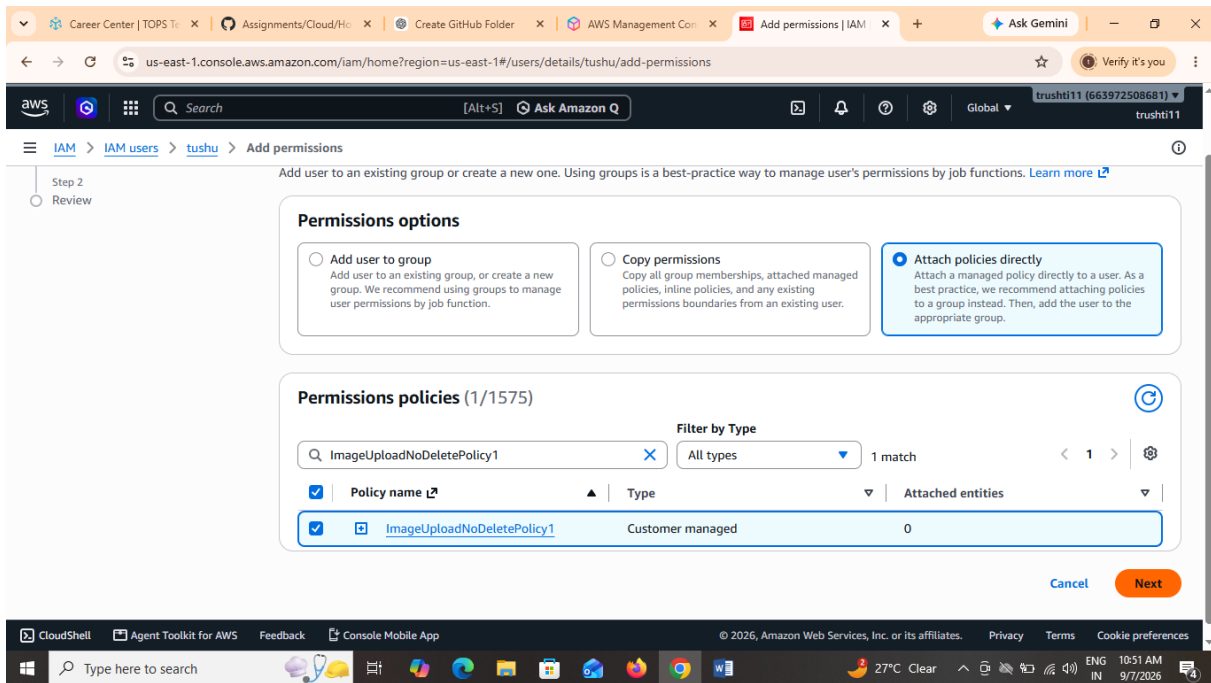


Figure 6 — Attaching ImageUploadNoDeletePolicy1 directly to the IAM user tushu.

Step 3 Review and create

The review screen confirms that user tushu will receive exactly one permissions policy, ImageUploadNoDeletePolicy1, used as its Permissions policy.

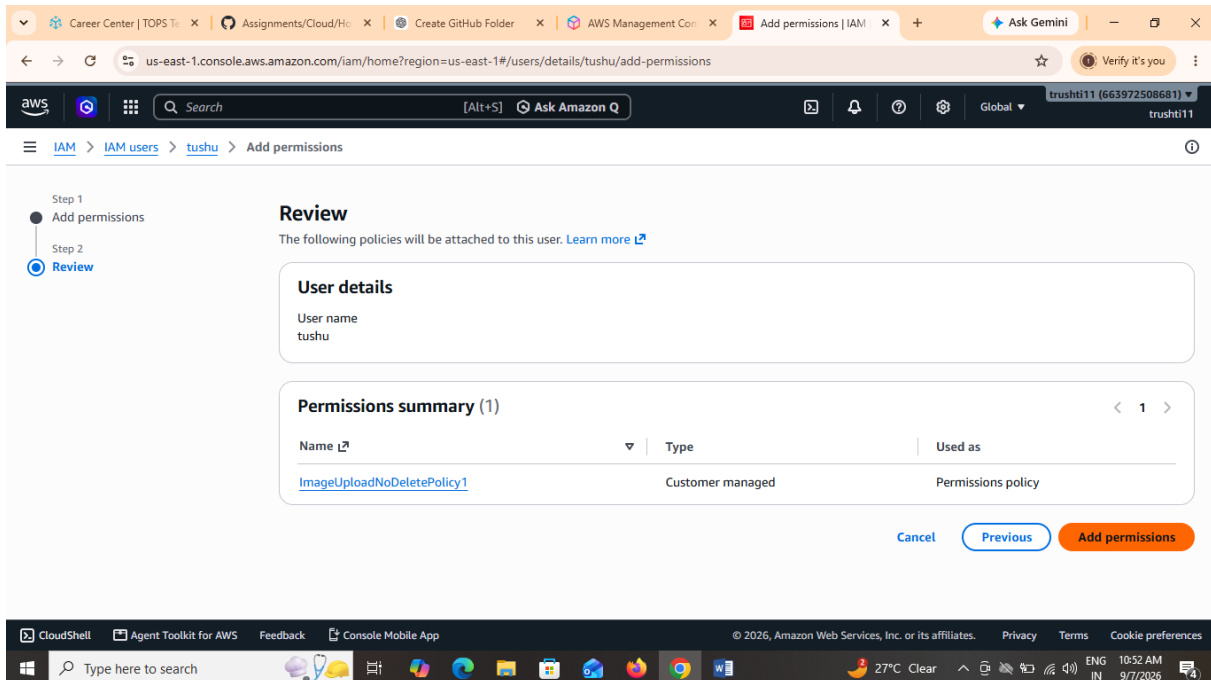


Figure 7 — Final review before the policy is attached to the user.

Important check

If a user already has a broad policy such as AmazonS3FullAccess attached, they will still be able to delete objects — an explicit Allow from any attached policy is enough, since this policy never issues an explicit Deny for DeleteObject. Always check IAM → Users → (user) → Permissions to confirm ImageUploadNoDeletePolicy1 is the only policy attached before testing.

10. Refining the Policy

While testing, the console listing of the bucket also needed the ability to see the account's bucket list, so the policy was revisited and an additional statement, `s3:ListAllMyBuckets` (scoped to all resources, since this action cannot be limited to one bucket), was added alongside the original two statements. The finalized policy attached to the user is shown below.

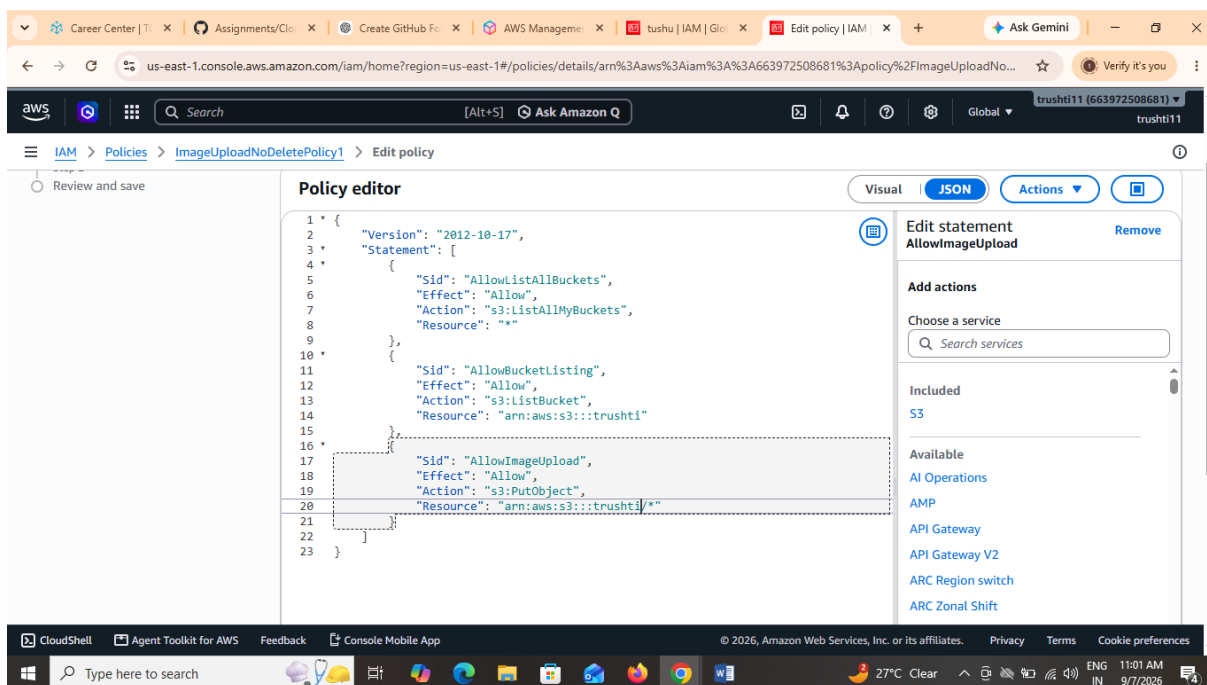


Figure 8 — Editing ImageUploadNoDeletePolicy1 to add AllowListAllBuckets alongside the original statements.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListAllBuckets",
      "Effect": "Allow",
      "Action": "s3:ListAllMyBuckets",
      "Resource": "*"
    },
    {
      "Sid": "AllowBucketListing",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::trushti"
```

```

    },
    {
      "Sid": "AllowImageUpload",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::trushti/*"
    }
  ]
}

```

11. Part 7 — Testing: Upload as the Restricted User

Signed in as the IAM user tushu (tested here in an incognito browser window to keep the session separate from the admin account), the bucket trushti is initially empty:

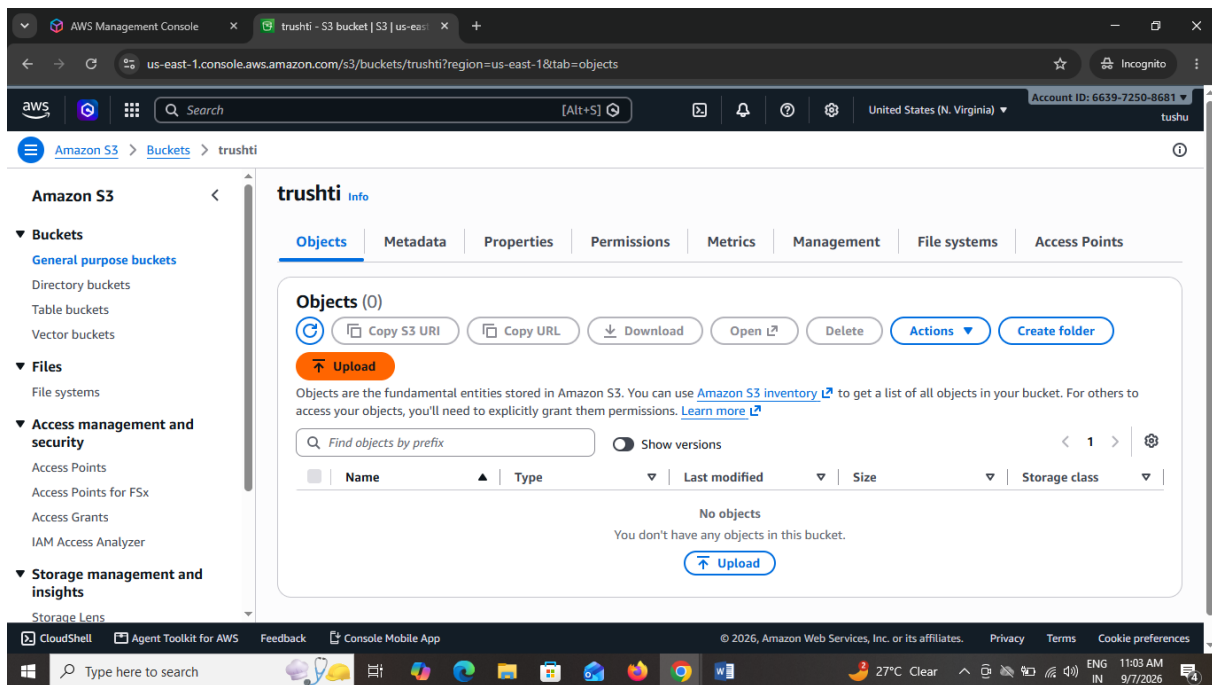


Figure 9 — Bucket trushti viewed as user tushu, showing 0 objects before the test.

Step 1 Select a file to upload

Click Upload, then Add files and choose a file from the local machine to act as the test object.

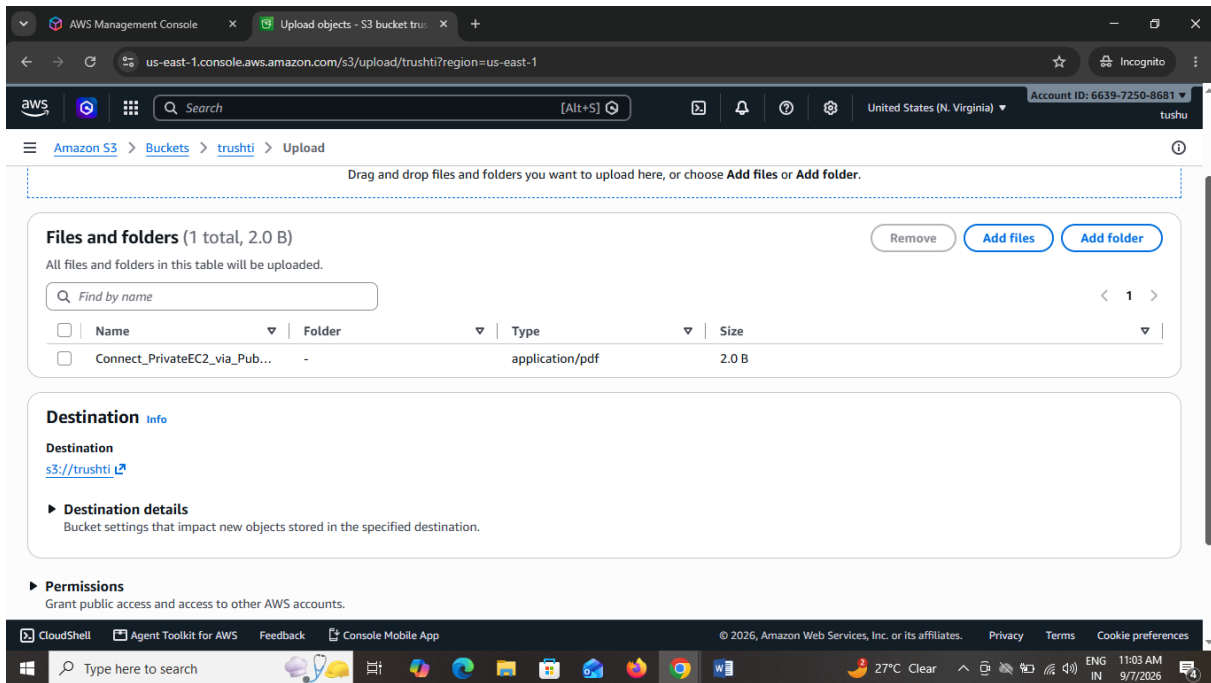


Figure 10 — A test file added to the upload queue, destination `s3://trushti`.

Step 2 Confirm the upload

Expand the Destination, Permissions, and Properties sections if needed, then click Upload.

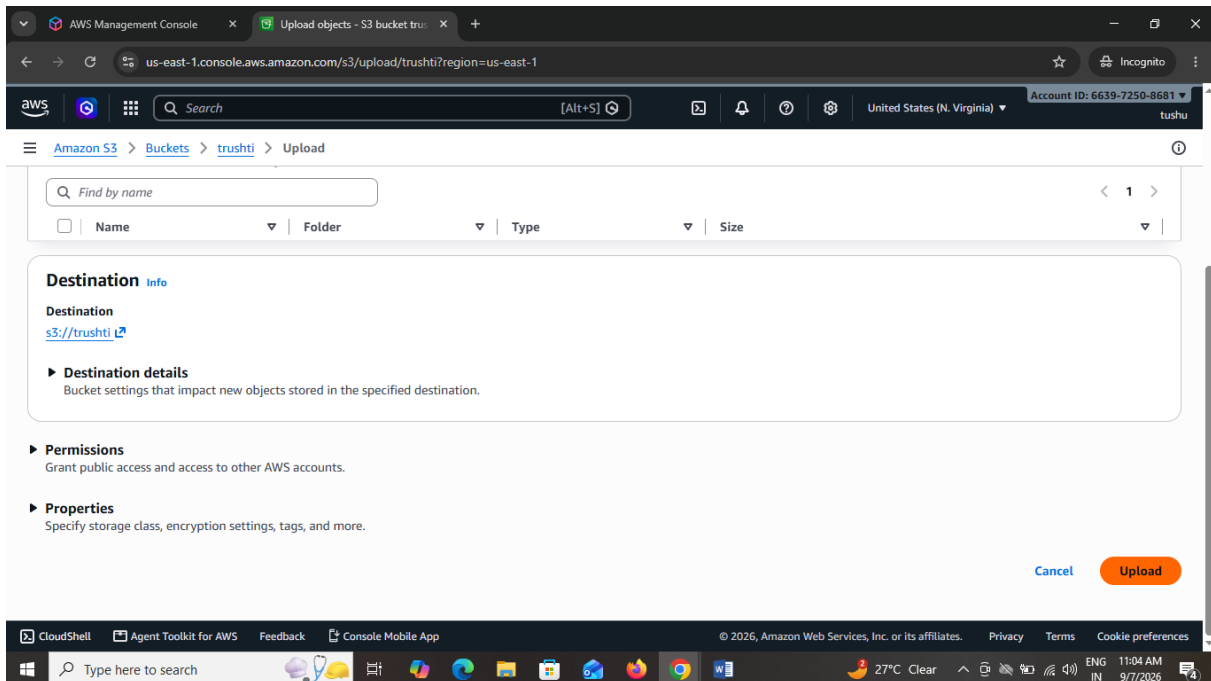


Figure 11 — Ready to upload — no public access is granted, keeping the bucket private.

Step 3 Verify the object appears in the bucket

The upload succeeds and the object is now listed in the bucket, confirming that `s3:PutObject` and `s3:ListBucket` are both working correctly for this user.

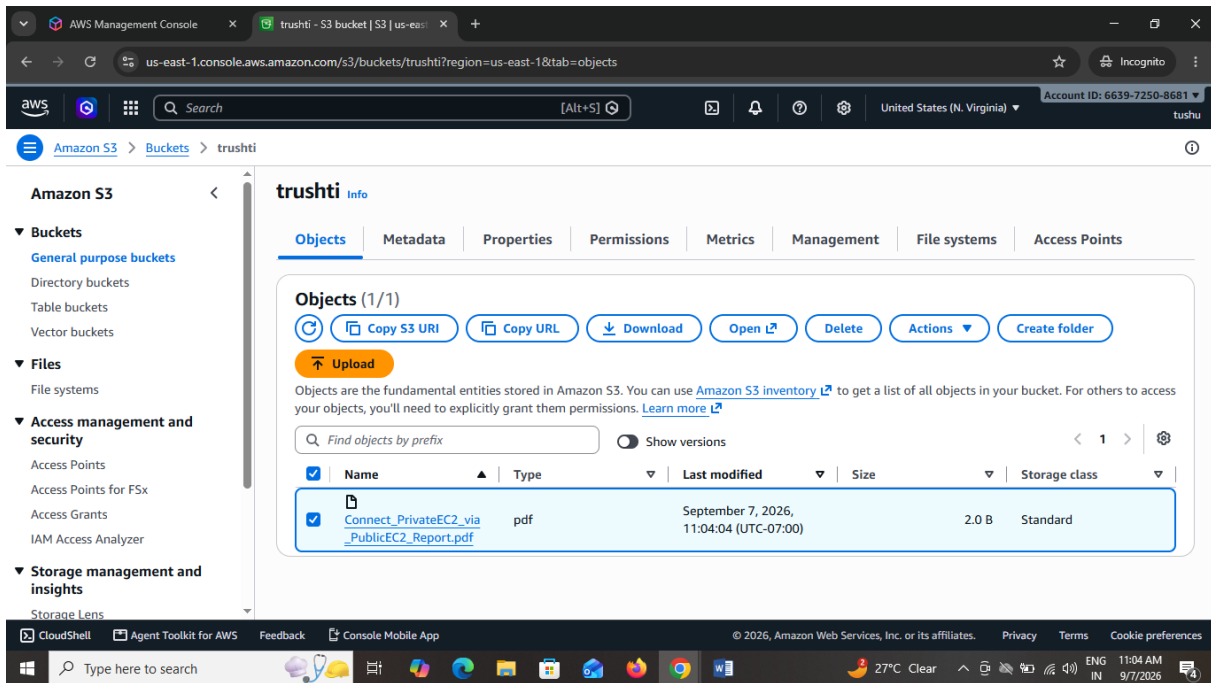


Figure 12 — Upload succeeded: the test object now appears in the trushti bucket listing.

12. Part 8 — Testing: Delete Should Be Blocked

With the same object selected, clicking Delete and confirming the action produces the expected failure:

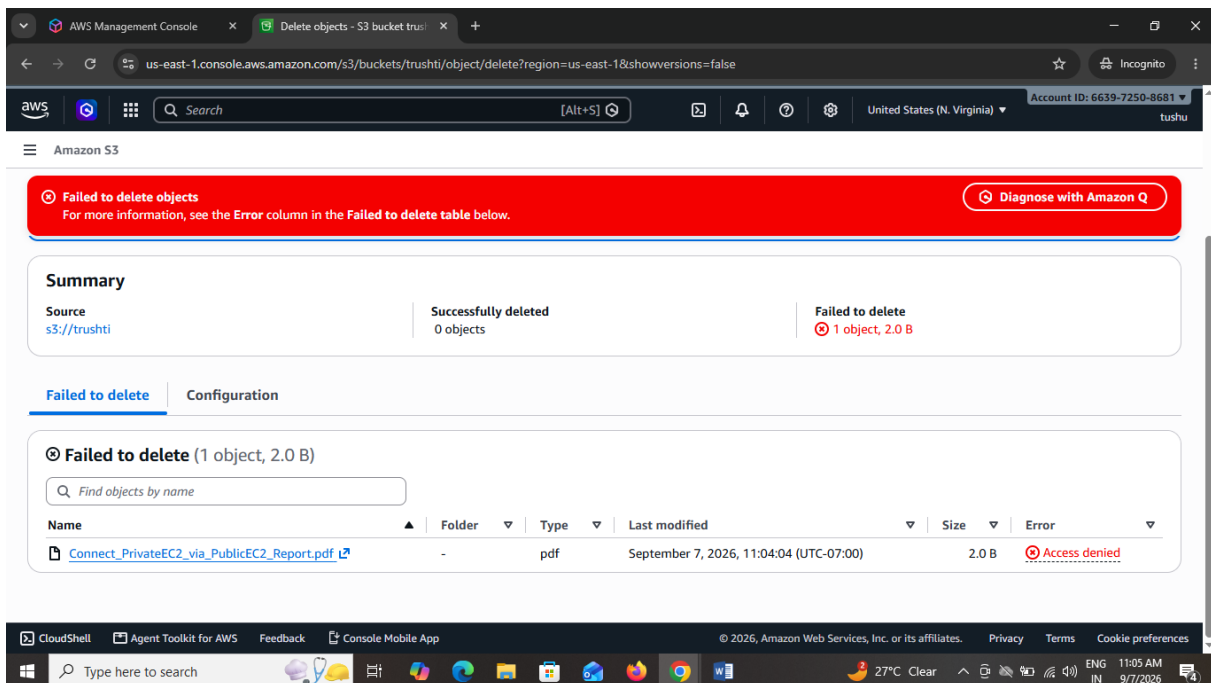


Figure 13 — Delete attempt fails with “Access denied”, proving `s3:DeleteObject` is not granted.

The console reports “Failed to delete objects” with an Access denied error against the object, and the summary confirms 0 objects were successfully deleted. This is exactly the expected behaviour: the user can upload and list, but cannot remove objects from the bucket.

13. Results Summary

Permission	Purpose	Result
s3:PutObject	Tested by uploading a file as user tushu	Success
s3:ListBucket	Tested by viewing the bucket's object list	Success
s3>DeleteObject	Tested by attempting to delete the uploaded object	Access Denied

14. Conclusion

This exercise demonstrates the principle of least privilege in AWS IAM: rather than granting broad access and trying to take permissions away, a custom policy was written that grants only the two actions actually needed — `s3:PutObject` and `s3:ListBucket` — and simply never mentions `s3>DeleteObject`. Because IAM's default behaviour is to deny anything not explicitly allowed, this omission is enough to block deletion outright, as confirmed by the Access Denied result in the final test. The same pattern can be extended to enforce upload-only access for any application or integration that should never be able to remove data from a bucket.